

A 3D rendering of the text "DL for ENG" in a bold, gold, sans-serif font. The letters are floating on a deep blue ocean with visible ripples and small waves. The sky above is a mix of light blue and grey, with soft, white clouds. The overall scene has a cinematic, slightly surreal quality.

DL for ENG

LECTURE 5.1

Convolutional and Graph Networks

Look only at the neighbours and reuse the weights: a convolution on a grid, message passing on a graph.

Remus Teodorescu

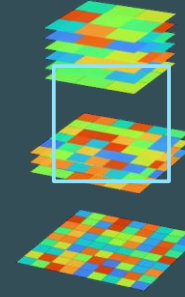
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

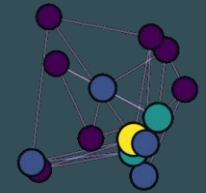
Deep Learning for Engineering · 2026

convolution · graph networks · the six-bus network

A GRID AND A GRAPH



a grid — every pixel has the same neighbours



a graph — whoever the edges connect

Look only at the neighbours and reuse the weights: a convolution on a grid, message passing on a graph.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Local rules** — why a dense layer is a poor fit for an image
- 2 **Convolution** — what a convolutional network is, and its main pieces
- 3 **Graphs** — the same idea on data made of connected parts

One small example runs through the lecture, a six-bus power network. You work with it again in the exercises.

WHAT WE SKIP, AND WHERE IT IS TAUGHT

residual networks	very deep networks — UDL ch. 11
graph convolution	one graph layer — UDL ch. 13.4
transfer learning	reusing a trained network — L6.2
camera network	the network on the car — L11.1
physics on a graph	physics in a grid network — L12.1
sequences	such as time series — L5.2

Recap

1 – 4 · DEPTH

- 1 What is a deep network, written as a composition, and what does one layer compute in matrix form? *UDL ch. 4.1 – 4.4*
- 2 Why does doubling the width cost four times the parameters while doubling the depth costs two? *UDL ch. 4.4; Ex_04 notebook 04*
- 3 Why do the linear regions multiply rather than add when layers are composed, and what does that say for depth? *UDL ch. 4.5*
- 4 What is the vanishing gradient, and which three changes made deep networks trainable? *UDL ch. 7.5, 11*

5 – 10 · GENERALISATION

- 5 What is model capacity, and how do training and held-out error behave as it grows? *UDL ch. 8.1*
- 6 A model passes through every training point. Why is that not good news, and how would you detect it? *UDL ch. 8.1; Ex_04 notebook 05*
- 7 What are the jobs of the training, validation and test sets, and why is the test set used once? *UDL ch. 8.1*
- 8 What are the three terms in the bias–variance decomposition, and what does an error below the noise floor suggest? *UDL ch. 8.2*
- 9 What does double descent add to the classical U-curve, and when does it matter in practice? *UDL ch. 8.4*
- 10 Given an overfitted model, why regularise before shrinking it, and which two methods would you try first? *UDL ch. 9.1; Ex_04 notebook 05*

Local Rules

Local rule — one small calculation that looks only at a point and its neighbours, with the same weights reused at every point. On a grid it is a convolution; on a graph it is message passing.

A DENSE LAYER ON AN IMAGE, AND WHAT REPLACES IT

- a **256 × 256 image** — 65,536 inputs, one per pixel
- a **dense layer of 1,000 neurons** — about 65 million weights
- the **problem** — to it, the pixels have no neighbours
- the **new rule** — a small window, the same weights everywhere
- one **3 × 3 kernel** — 10 parameters, for any image size
- the **lesson** — build in what we know about the data

ON A GRID — CONVOLUTION

computes	a weighted sum over a small window
parameters	the kernel weights and one bias
reused	at every position, so for any image size

ON A GRAPH — MESSAGE PASSING

computes	a sum over the connected nodes
parameters	one weight matrix and one bias
reused	at every node, so for any graph size

Convolutional Network

Convolutional network — a network for data laid out on a grid, such as an image. Each layer looks at small neighbourhoods and reuses the same weights at every position.

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a feed-forward network whose layers are convolutions
- **function** — image in, features or label out, shift-equivariant
- **architecture** — convolution and pooling, then a dense head
- **applications** — classification, segmentation, detection, 1-D signals

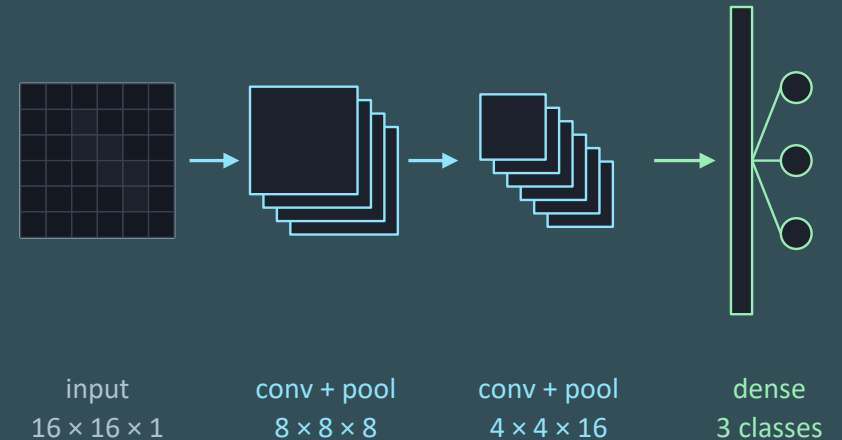
AGAINST A DENSE NETWORK

	dense network	convolutional
connections	every input to every neuron	a small window only
weights	one per connection	one kernel, reused at every position
parameters	grow with the input size	weights and biases, any image size
a shifted input	an unrelated output	the same shift of the output
input size	fixed when designed	any, with global pooling

CONVOLUTION AND NETWORK

$$h_{ij} = a \left(\sum_m \sum_n k_{mn} x_{i+m, j+n} + b \right)$$

x the input image, k_{mn} the kernel, b one bias
 a the activation, h_{ij} one output pixel



Ex05 notebook 01 · 2,019 parameters · clean, crack, pit

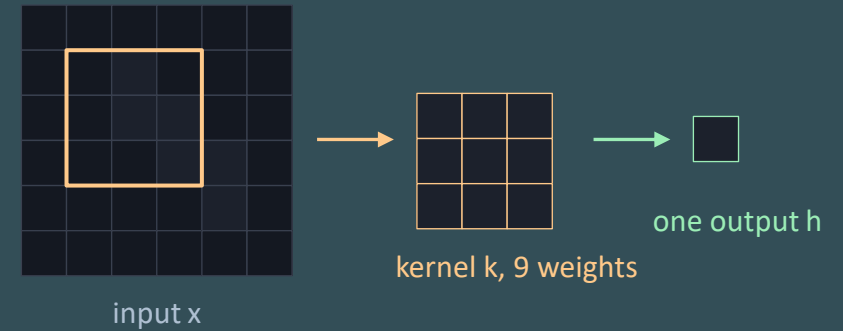
Kernel

Kernel — the small set of weights a convolution slides over its input. At each position it takes a weighted sum of the neighbouring values, adds a bias and applies the activation.

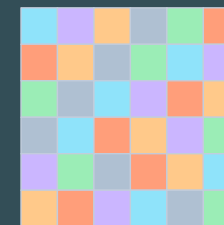
A WEIGHTED SUM OVER A SMALL WINDOW

- a **3 × 3 window** — multiply, sum, add a bias, apply the activation
- **move one pixel and repeat** — with the same nine weights
- **the nine weights are learned** — once set by hand, now trained
- **the output** — an image of the same kind, one value per position
- **an edge detector** — weights -1, 0, +1 across the columns

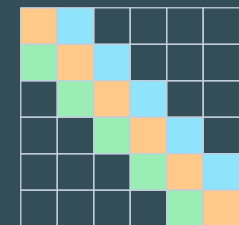
ONE WINDOW, ONE OUTPUT PIXEL



THE SAME IDEA AS A WEIGHT MATRIX



dense
36 weights



convolution
3 weights, reused

Convolution Settings

Stride, padding and channels — the choices that fix a convolution layer's shape: how far the kernel moves each step, how the border is filled, and how many feature maps go in and come out.

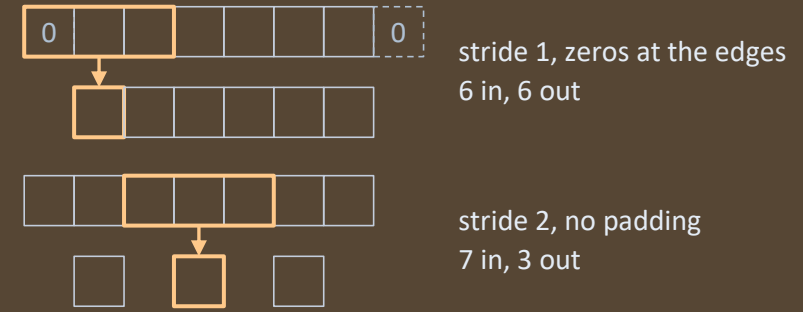
CHOSEN BEFORE TRAINING, THEN LEARNED

- **stride** — the step the window takes; a step of 2 halves the image
- **padding** — a border of zeros, so the output keeps the input's size
- **channels** — feature maps, each with its own kernel
- **parameters** — kernel area $\times$ maps in $\times$ maps out, plus biases

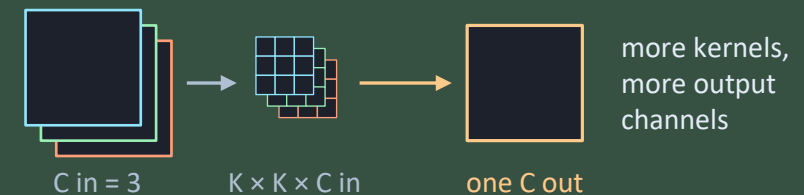
$$P_{\text{dense}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}} \quad P_{\text{conv}} = K^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$$

$N_{\text{in}}, N_{\text{out}}$ neurons in and out, K the kernel side, $C_{\text{in}}, C_{\text{out}}$ channels in and out

STRIDE AND PADDING, IN 1D



CHANNELS, AND ONE LAYER COUNTED



64 $\rightarrow$ 64 channels, 3 $\times$ 3: $64 \times 64 \times 9 + 64 = 36,928$

1 $\rightarrow$ 1 channel: $9 + 1 = 10$

Pooling and Hierarchy

Pooling — a step that shrinks a feature map by summarising each small block, for example by its largest value. Stacked with convolutions, it builds features from edges up to whole objects.

SHRINK THE IMAGE, BUILD UP FEATURES

- **pooling** — each 2×2 block becomes its largest value
- **why pool** — less computation; small shifts matter less
- **image shrinks, channels grow** — less where, more what
- **the hierarchy** — edges, textures, parts, whole objects
- **nobody designs it** — it emerges from training
- **when not to pool** — when the answer must say where

MAX POOLING, 2×2

1	3	2	0
4	2	1	1
0	1	5	2
2	0	3	4

input, 4×4

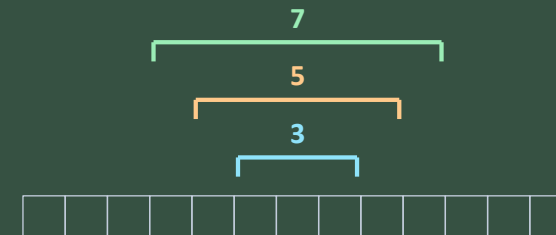


4	2
2	5

output, largest of each

WHAT ONE NEURON SEES, BY DEPTH

each layer widens the reach



layer 3 · parts
layer 2 · textures
layer 1 · edges

Equivariance

Equivariance — shift the input and the output shifts the same way.
 Invariance: shift the input and the output does not change. Convolution is equivariant; pooling adds invariance.

EQUIVARIANT AND INVARIANT

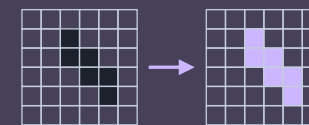
- **equivariant** — move the input, and the output moves with it
- **invariant** — move the input, and the answer stays the same
- **for example** — the crack map moves; “crack: yes” does not
- **convolution** — equivariant: the same kernel everywhere
- **pooling** — adds invariance by throwing position away

TALK · 2 MIN How do you understand convolution — what does it do to an image, in your own words? Then, in WHAT EACH OPERATION GIVES on the right, which row describes a crack detector that must say where the crack is, and which a classifier that only says whether there is one?

WHAT EACH OPERATION GIVES

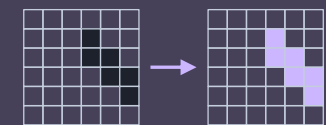
	property	used for
convolution	equivariant	every layer
pooling	partly invariant	classification
global average	invariant	one label per image
no pooling	equivariant	segmentation

input, and its crack map



crack: yes

moved one pixel right



crack: yes

the map moves: equivariant · the answer stays: invariant

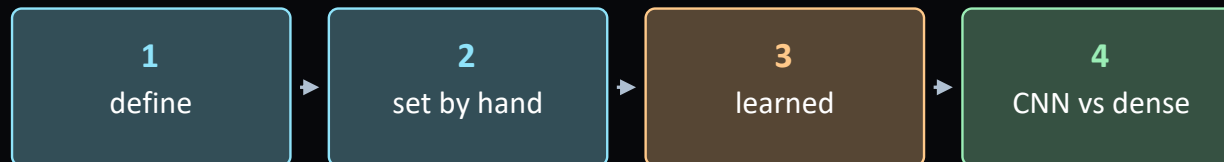
Convolution in Practice

A convolutional network in practice — a few convolution and pooling layers, then a dense head, trained on labelled images. In Ex05 notebook 01: weld radiographs of clean plate, cracks and pits.

WHAT THE CONVOLUTION SECTION SHOWED

- **local and shared** — small window, same weights everywhere
- **few parameters** — kernel weights and biases, for any image size
- **settings** — stride, padding and channels, chosen before training
- **pooling** — shrinks the image; deeper layers see more of it
- **equivariant** — output follows input; pooling adds invariance

IN EX05 NOTEBOOK 01



NOTEBOOK 01, THE RESULT

	parameters	train	held out
convolutional	2,019	1.000	0.989
dense	16,643	0.995	0.711

600 weld radiographs, 16×16 · clean, crack, pit

THE SAME RESULT IN UDL

convolutional about 17 % test error

dense about 40 % test error

MNIST-1D · same depth and width · structure built in wins

Graphs

Graph — a set of nodes joined by edges. The nodes carry values, the edges say which nodes are connected, and there is no grid and no natural order.

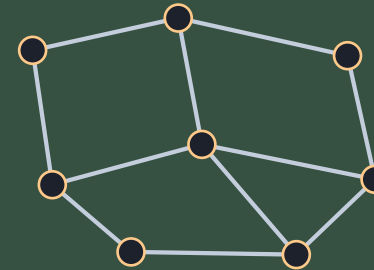
NODES AND EDGES, IN FOUR FIELDS

	nodes	edges
road map	junctions	roads between them
molecule	atoms	chemical bonds
electrical circuit	junctions	the components
power grid	buses	transmission lines

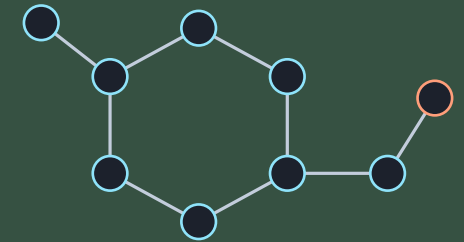
THE TYPES WE USE

- **undirected** — a line carries power both ways
- **directed** — one way only, like one paper citing another
- **edge features** — numbers on each edge, like line admittance

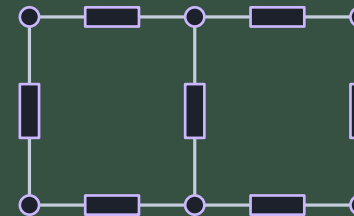
FOUR GRAPHS



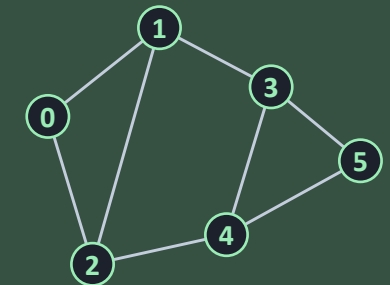
road map



molecule



electrical circuit



power grid, six buses

Graph Representation

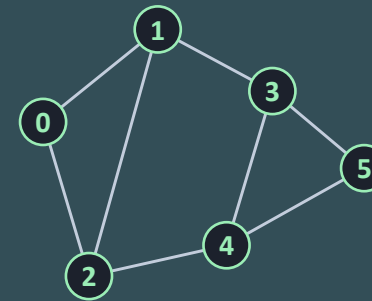
Graph representation — three arrays: X , one row of features per node; A , the adjacency matrix, with a one wherever two nodes are joined; and E , one row of features per edge.

THREE MATRICES DESCRIBE THE WHOLE GRAPH

- **node features X** — one row per node: what is measured there
- **adjacency A** — a one wherever two nodes are connected
- **edge features E** — one row per edge: here, the line admittance
- **sparse** — 16 non-zeros of 36; real grids are far sparser

TALK · 2 MIN How would you explain a graph to someone outside engineering, and what makes a power grid one? Then, in the adjacency matrix on the right, which row belongs to bus 2, and which three columns of it are lit?

THE SIX BUSES AS MATRICES



	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

$A, 6 \times 6$

	P	Q	V	S	PV	PQ
0						
1						
2						
3						
4						
5						

$X, 6 \times 6$

0-1	16.5
0-2	14.2
1-2	16.5
1-3	12.4
2-4	14.2
3-4	16.5
3-5	9.9
4-5	12.4

$E, 8 \times 1, |y|$

Graph Neural Network

Graph neural network — a network whose input is a graph. Each layer updates every node from its neighbours with one shared rule, so the same network works for any number of nodes.

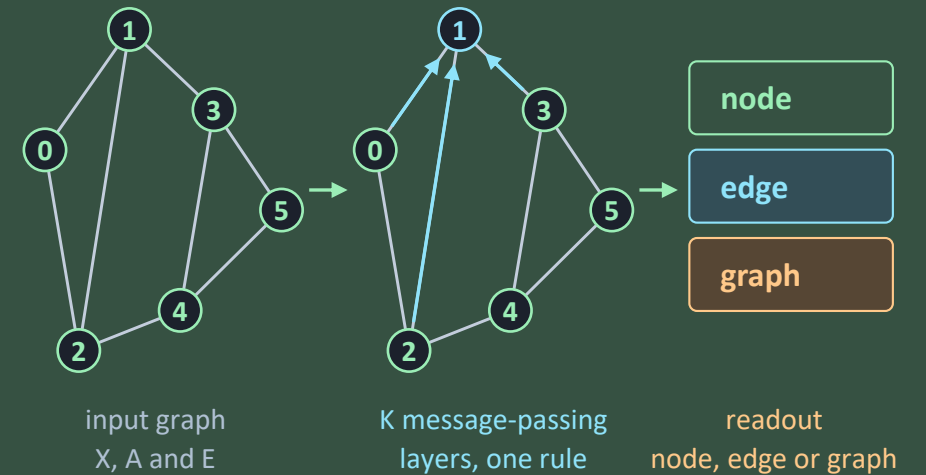
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — layers that update every node from its neighbours
- **function** — a graph in, X, A and E; an answer out
- **architecture** — K message-passing layers, then a readout
- **applications** — power grids, circuits, molecules, road maps

AGAINST A DENSE NETWORK

	dense network	graph network
input	one vector, fixed length	a graph of any size, with features
connections	every input to every neuron	along the edges only
weights	one per connection	one rule, shared by every node
renumbered nodes	an unrelated output	the same output, renumbered
topology	fixed when trained	an input; one model, many graphs

THE ARCHITECTURE, ON THE SIX BUSES



bus 1 gathers from buses 0, 2 and 3 · every bus does the same, with the same rule

Graph Tasks and Losses

Graph task — what a graph network predicts: a value for each node, for each edge, or one answer for the whole graph. Each is trained with the loss that suits it.

WHAT IS PREDICTED, AND HOW IT IS SCORED

task	output	loss
node regression	a number per node	squared error, as in notebook 03
node classification	a class per node	cross-entropy
edge prediction	a probability per edge	binary cross-entropy
graph classification	one class per graph	cross-entropy, on the averaged nodes

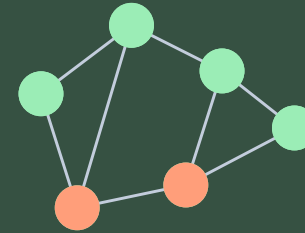
$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}$$

N nodes, C classes, $y_{ic} = 1$ for the true class, $\hat{y}_{ic}$ the predicted probability

A NEW GRAPH, OR THE SAME ONE

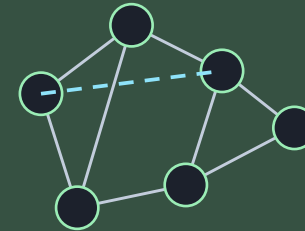
- **inductive** — train on some graphs, predict on graphs never seen
- **a tripped line** — a new graph; notebook 03 predicts it directly
- **transductive** — one fixed graph; label its remaining nodes

THREE LEVELS OF ANSWER



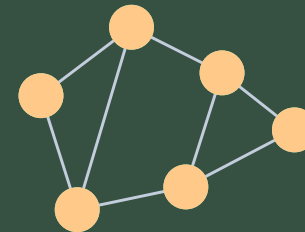
node

a value at every bus, as in notebook 03



edge

a probability for each line, such as whether it is present



graph

one answer for the whole grid, such as whether it is stable

Message Passing

Message passing — one layer of a graph network: every node collects messages from its neighbours, sums them, and combines the sum with its own state to get its new state.

THREE STEPS, REPEATED AT EVERY NODE

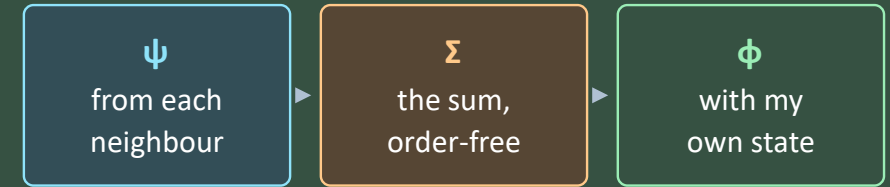
- **for each node** — compute a message from each neighbour
- **sum the messages** — then combine with the node's own state
- **both functions are small networks** — shared across every node
- **the sum** — ignores order and count: one model, any graph size
- **K layers reach K hops** — match K to the graph diameter

$$\mathbf{h}_i^{(l+1)} = \phi \left(\mathbf{h}_i^{(l)}, \sum_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}) \right)$$

$\mathbf{h}_i^{(l)}$ node i at layer l , $\mathcal{N}(i)$ its neighbours, ψ the message, ϕ the combine

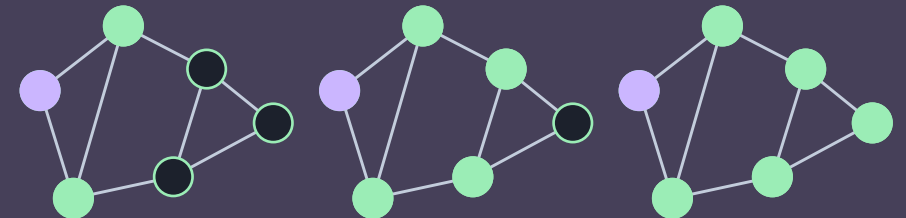
the physics sets the depth, weights the edges, gives the ground truth

ONE NODE'S UPDATE



same functions at every node · repeat for more hops

HOW FAR BUS 0 HEARS, LAYER BY LAYER



layer 1: 3 of 6 buses

layer 2: 5 of 6 buses

layer 3: 6 of 6 buses

diameter 3: three layers reach every bus

Permutation Equivariance

Permutation equivariance — renumber the nodes and the output is renumbered the same way, and nothing else changes. A graph network has this property by construction.

RENUMBER THE NODES, THE OUTPUT FOLLOWS

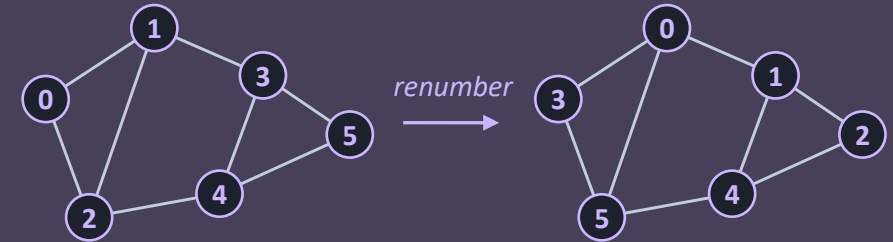
- **bus numbers** — bookkeeping, with no natural order
- **an answer that changes with them** — is wrong
- **a graph network** — the output follows exactly
- **a dense network** — no such guarantee

$$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^T) = \mathbf{P} f(\mathbf{X}, \mathbf{A})$$

$\mathbf{X}$ node features, $\mathbf{A}$ the adjacency, $\mathbf{P}$ any relabelling of the nodes, f the network

TALK · 2 MIN Why must a grid model give the same answer however the buses are numbered? Then, in the figure on the right, bus 0 is relabelled 3: where must its row of the output go?

ONE GRID, TWO NUMBERINGS



same lines, same physics · only the labels changed

GRAPH AND DENSE, RENUMBERED

	graph network	dense network
renumber input	output renumbered	a different answer
because	one shared rule, a sum	a weight per position
input size	any number of buses	fixed when trained

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
residual networks	skips let very deep networks train	UDL ch. 11	image recognition at scale
batch normalisation	rescales outputs between layers	UDL ch. 11.4	very deep image networks
graph convolution	neighbours weighted by degree	UDL ch. 13.4	citation, social networks
graph attention	learns how much a neighbour counts	UDL ch. 13.8	molecules, recommendations
encoder–decoder	shrinks, then regrows the image	UDL ch. 10.5	medical image segmentation

WHY THEY ARE LEFT OUT

Each of these earns its place on very large data sets: millions of photographs, molecules or social-network users. The problems in this course are small and physical, and today's convolution and message passing are enough for them.

Exercise 5

NOTEBOOKS 01 TO 03, AND 04 AFTER L5.2

- **01** — cracks and pits in weld radiographs, with a CNN
- **02** — graphs from scratch: adjacency and message passing
- **03** — a graph network predicts the six-bus power flow
- **04** — a sequence task, after the next lecture



THE QUESTION EACH NOTEBOOK ANSWERS

- 01** does a CNN beat a dense network with far fewer parameters?
- 02** how does a node gather from its neighbours, and why is the numbering irrelevant?
- 03** how close does it come to the AC power flow, how much faster, and after a trip?

REPORT QUESTION

The graph network answers far faster than the AC power flow, and less exactly. When is the speed worth the error, before and after a line trips?

Key Takeaways

1 What kind of network

Convolutional: a feed-forward network whose layers are convolutions. Graph: layers that update every node from its neighbours.

2 What function it implements

A grid in, features or a label out, shift-equivariant. A graph in, an answer per node, edge or graph, permutation-equivariant.

3 What the architecture is

Convolution and pooling, then a dense head. K message-passing layers, then a readout.

4 How it differs from a dense network

Local connections and one shared rule: few parameters, any input size, and a symmetry built in.

5 What it is used for

Images and 1-D signals: classification, segmentation, detection. Power grids, circuits, molecules, road maps.

Next — L5.2: RNN and LSTM.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.3*
- 2 What does a 3×3 kernel compute at one position, what does a $-1, 0, +1$ kernel find, and why are kernels learned? *UDL ch. 10.2; Ex_05 notebook 01*
- 3 How many parameters has a convolution layer, and what do stride, padding and channels each change? *UDL ch. 10.2 – 10.3*
- 4 Describe a convolutional network layer by layer: what does pooling do, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.4*
- 5 How does a convolutional network differ from a dense network, and why does it do better on unseen images with far fewer parameters? *UDL ch. 10.2; Ex_05 notebook 01*
- 6 What is a graph, and how are the six buses and eight lines written as X, A and E? *UDL ch. 13.1 – 13.2; Ex_05 notebook 02*
- 7 What kind of network is a graph network, what goes in and comes out, and how does it differ from a dense network? *UDL ch. 13.3*
- 8 Describe one message-passing layer. Why does the sum suit any number of neighbours, and how many layers reach every bus? *UDL ch. 13.4, 13.8; Ex_05 notebook 02*
- 9 What does permutation equivariance guarantee, and how would you check it for a model? *UDL ch. 13.2; Ex_05 notebook 02*
- 10 What is predicted at node, edge and graph level, and why can a graph network take a tripped line without retraining? *UDL ch. 13.3, 13.6; Ex_05 notebook 03*

NEXT

L5.2 — RNN and LSTM

Today's structures were spatial, a grid and a graph. Next time the structure is temporal: how to carry information from the start of a long sequence to its end.

BEFORE THE NEXT SESSION

read GBC ch. 10.1 – 10.2, 10.10

run Ex_05 notebooks 00 to 03

bring your tripped-line result

think about layer count against graph diameter

both networks' errors, intact grid and after the trip, and the speed